



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

MONITORING OF BITTORRENT TRAFFIC IN LAN

PROJECT REPORT

AUTHOR

SAMUEL OLEKŠÁK

BRNO 2023

Contents

1	Introduction	2
2	BitTorrent architecture and communication	2
2.1	Distributed hash table (DHT)	2
2.2	Bencoding	2
3	Exploring BitTorrent clients	2
4	Methods for BitTorrent traffic detection	3
4.1	Bootstrap node detection	3
5	Implemented application	3
6	Testing and evaluation	4
7	Results	4
8	Conclusion and contribution	4
	Bibliography	5

1 Introduction

This project gives a short overview of Bittorrent network; proposes and evaluates methods for detection and monitoring of Bittorrent traffic on the network; and provides an application for detecting bootstrap nodes, peers and file transfers in captured network traffic.

2 BitTorrent architecture and communication

BitTorrent is a peer-to-peer (P2P) protocol for decentralized file sharing over the Internet.

Shared file is distributed in small chunks called *pieces*. Size of the pieces is not constant and can vary. Common chunk size is 256 KB [2].

[[swarm, peer, leech, seed, tracker]]

2.1 Distributed hash table (DHT)

Bittorrent network uses two main approaches for searching for files on the network. In case trackers for desired files are known, they can be queries for the location of the file. In case there is no known tracker for a file, DHT protocol [1] is used.

2.2 Bencoding

Bencoding [2] is an encoding format used by Bittorrent protocol. It supports encoding of integers, strings, lists and dictionaries.

As an example, the bencoded string `d6:lengthi16e4:name7:PDS.txt12:piece lengthi16384ee` would be decoded into the following dictionary:

```
{
  length: 16,
  name: PDS.txt,
  piece length: 16384
}
```

3 Exploring BitTorrent clients

[[Your experiments with BitTorrent client and findings.]] For the experiments two Bittorrent clients were used – *qBittorrent*¹ and *Bittorrent classic*². Each experiment result was captured and saved into packet capture file:

- `installation.pcapng` – Captured packets from the first ever startup of the client after installation.
- `startup.pcapng` – Captured packets from the subsequent startup of the client.
- `small-file.pcapng` – Client startup, downloading and seeding of a small file.

Every file has two instances – one in the `qbittorrent` and another in the `bittorrent-classic` folders. Capture files were cleaned up applying the display filter `not arp` and `not nat-pmp`.

¹<https://www.qbittorrent.org/>

²<https://www.bittorrent.com/products/win/bittorrent-classic-free>

4 Methods for BitTorrent traffic detection

4.1 Bootstrap node detection

Using a single methods for bootstrap node detection has proven to be unreliable across different clients, therefore multiple detection techniques with their own pros and cons are used to patch each other weaknesses. Bootstrap nodes are extracted from packets containing BT-DTH protocol [1], which is detected by trying to apply bdecoding on every UDP packet. If bdecoding is successful, it is assumed that packet contains BT-DTH protocol³.

bs attribute detection

qBittorrent client makes detecting bootstrap straight-forward by including `bs:1` in the request arguments dictionary in `get_peers` requests. This behaviour was not observed in *Bittorrent classic*.

DNS IP address matching

Bootstrap nodes are often⁴ hardcoded in source codes of torrent clients in the form of domain names. IP addresses received from translation of these domain names through DNS communication can saved. If a DHT-BT communication takes place between client's IP address and IP address received from DNS, there is high likelihood that node is a bootstrap node. Con of this approach is that some clients could cache DNS responses, which would render this method inconsistent, although this behaviour was observed by neither of the Bittorrent clients.

Some nodes could theoretically be falsely labelled bootstrap if they are received in the form of a domain name instead of IP address, which does not occur in standard BT-DHT.

Well known DNS list and keyword detection

[[joining response and request, first receive then send]]

5 Implemented application

[[Description of implemented application, i.e., how application operates, how is compiled, what parameters expects, how can be run.]] Application was implemented in Python. Scapy library is used to parse packet capture files. Application expects pcap files to be supplied using the `-pcap` parameter. No `tshark` preprocessing into CSV format is needed.

Detailed instructions on how to run the application are printed upon providing either `-h` or `-help` parameter.

³It is certainly possible that a non-standard or non-BT-DTH UDP packet containing bencoded payload is encountered so robust error checking has to be established for non-conformant bencoded payloads.

⁴See source code of *Deluge* client (<https://dev.deluge-torrent.org/browser/deluge/core/preferencesmanager.py?rev=415979e2f76658c4e325b7854f0a66206f0bc5c8#L320>) or *qBittorrent* client (<https://github.com/qbittorrent/qBittorrent/blob/7397c80837eddd4987c46dd30dc3e514c1213c58/src/base/bittorrent/sessionimpl.cpp#L1853>)

6 Testing and evaluation

[[Testing of the application on created datasets, results, evaluation]]

7 Results

8 Conclusion and contribution

Due to the often non-standard and inconsistent nature of Bittorrent clients, which is likely a consequence of deliberate decentralization, multiple methods need to be interleaved to provide somewhat robust architecture for Bittorrent traffic detection across the vast spectrum of available clients.

The goal of this project was to explore architecture of Bittorrent architecture and how various clients communicate with the Bittorrent network during initialization, downloading and seeding of files; propose methods for detecting Bittorrent traffic in network communication; implement and describe an application for performing these methods on packet capture files; and evaluate these methods with regard to reliability across different Bittorrent clients.

Bibliography

- [1] ANDREW LOEWENSTERN, A. N. *DHT protocol*
[https://www.bittorrent.org/beps/bep_0005.html]. January 2008. Accessed: April 16, 2023.
- [2] COHEN, B. *The BitTorrent Protocol Specification*
[http://www.bittorrent.org/beps/bep_0003.html]. January 2008. Accessed: April 16, 2023.